HPC2

Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use

existing algorithms and measure the performance of sequential and parallel algorithms.

```
#include <iostream>     // for input-output (cout)
#include <vector>       // for dynamic arrays (vector)
#include <cstdlib>      // for rand()
#include <ctime>        // for time()
#include <omp.h>        // for OpenMP (parallel programming)

using namespace std;

#define SIZE 10000      // size of array (large to show performance difference)

// 🔹 Generate Random Data
void generateRandom(vector<int>& arr) {
    // fill array with random values (0 to 99999)
    for (int &x : arr)
        x = rand() % 100000;
}

// ================================
// 🔵 Sequential Bubble Sort
// ================================
void bubbleSortSeq(vector<int>& arr) {
    int n = arr.size();   // get size of array
```

```
    // outer loop → number of passes
    for (int i = 0; i < n - 1; i++) {

        // inner loop → compare adjacent elements
        for (int j = 0; j < n - i - 1; j++) {

            // if elements are in wrong order → swap
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
        }
    }
}

// ================================
// 🔵 Parallel Bubble Sort
// (Odd-Even Transposition Sort)
// ================================
void bubbleSortParallel(vector<int>& arr) {
    int n = arr.size();

    // perform n phases
    for (int i = 0; i < n; i++) {

        // 🔸 EVEN PHASE
        // compare (0,1), (2,3), (4,5) in parallel
        #pragma omp parallel for
        for (int j = 0; j < n - 1; j += 2) {
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
```

```
        }

        // 🔸 ODD PHASE
        // compare (1,2), (3,4), (5,6) in parallel
        #pragma omp parallel for
        for (int j = 1; j < n - 1; j += 2) {
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
        }
    }
}

// ================================
// 🔴 Merge Function
// ================================
void merge(vector<int>& arr, int l, int m, int r) {

    int n1 = m - l + 1;   // size of left subarray
    int n2 = r - m;       // size of right subarray

    vector<int> L(n1), R(n2);   // temporary arrays

    // copy data into left array
    for (int i = 0; i < n1; i++)
        L[i] = arr[l + i];

    // copy data into right array
    for (int j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];
```

```
    int i = 0, j = 0, k = l;

    // merge both arrays in sorted order
    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            arr[k++] = L[i++];   // take from left
        else
            arr[k++] = R[j++];   // take from right
    }

    // copy remaining elements (if any)
    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];
}

// ================================
// 🔴 Sequential Merge Sort
// ================================
void mergeSortSeq(vector<int>& arr, int l, int r) {

    if (l < r) {   // base condition

        int m = (l + r) / 2;   // find middle

        // recursively sort left half
```

```
        mergeSortSeq(arr, l, m);

        // recursively sort right half
        mergeSortSeq(arr, m + 1, r);

        // merge both halves
        merge(arr, l, m, r);
    }
}

// ================================
// 🔴 Parallel Merge Sort
// ================================
void mergeSortParallel(vector<int>& arr, int l, int r) {

    if (l < r) {

        int m = (l + r) / 2;   // find middle

        // run left and right halves in parallel
        #pragma omp parallel sections
        {
            #pragma omp section
            mergeSortParallel(arr, l, m);   // left part

            #pragma omp section
            mergeSortParallel(arr, m + 1, r); // right part
        }
```

```
        // merge results
        merge(arr, l, m, r);
    }
}

// ================================
//  🔹 MAIN FUNCTION
// ================================
int main() {

    vector<int> arr(SIZE), temp;   // main array + temporary copy

    srand(time(0));        // initialize random seed
    generateRandom(arr);   // fill array with random numbers

    double start, end;     // for measuring time

    // 🔵 Sequential Bubble Sort
    temp = arr;   // copy original data
    start = omp_get_wtime();   // start time
    bubbleSortSeq(temp);       // call function
    end = omp_get_wtime();     // end time
    cout << "Sequential Bubble Sort Time: " << (end - start) << " sec\n";

    // 🔵 Parallel Bubble Sort
    temp = arr;
    start = omp_get_wtime();
    bubbleSortParallel(temp);
```

```
    end = omp_get_wtime();
    cout << "Parallel Bubble Sort Time: " << (end - start) << " sec\n";

    // 🔴 Sequential Merge Sort
    temp = arr;
    start = omp_get_wtime();
    mergeSortSeq(temp, 0, SIZE - 1);
    end = omp_get_wtime();
    cout << "Sequential Merge Sort Time: " << (end - start) << " sec\n";

    // 🔴 Parallel Merge Sort
    temp = arr;
    start = omp_get_wtime();
    mergeSortParallel(temp, 0, SIZE - 1);
    end = omp_get_wtime();
    cout << "Parallel Merge Sort Time: " << (end - start) << " sec\n";

    return 0;   // program ends
}

/*
g++ -fopenmp -std=c++11 hpc2.cpp -o hpc2
hpc2.exe
 */
```

📘 Program Explanation (Line by Line)

🔹 Header Files

#include <iostream>

👉 Used for input/output (cout)

#include <vector>

👉 Used for dynamic array (vector)

#include <cstdlib>

👉 Used for random number generation (rand())

#include <ctime>

👉 Used to initialize random seed (time(0))

#include <omp.h>

👉 Enables OpenMP for parallel execution

🔹 Namespace

using namespace std;

👉 Avoid writing std:: repeatedly

🔹 Define Size

#define SIZE 10000

👉 Size of array = 10,000 elements
👉 Large size helps show performance difference

🔹 Random Array Generation

void generateRandom(vector<int>& arr)

👉 Function to fill array with random numbers

```
for (int &x : arr)
    x = rand() % 100000;
```

👉 Assign random values between 0–99999

🔵 Sequential Bubble Sort

```
void bubbleSortSeq(vector<int>& arr)
```

👉 Normal bubble sort function

```
int n = arr.size();
```

👉 Get size of array

```
for (int i = 0; i < n - 1; i++)
```

👉 Outer loop (number of passes)

```
for (int j = 0; j < n - i - 1; j++)
```

👉 Inner loop for comparison

```
if (arr[j] > arr[j + 1])
    swap(arr[j], arr[j + 1]);
```

👉 Swap if elements are in wrong order

🔵 Parallel Bubble Sort

```
void bubbleSortParallel(vector<int>& arr)
```

👉 Parallel version using OpenMP

🔸 Outer Loop

```
for (int i = 0; i < n; i++)
```

👉 Runs n phases (sorting passes)

🔸 Even Phase

```
#pragma omp parallel for
for (int j = 0; j < n - 1; j += 2)
```

👉 Parallel loop
👉 Compare pairs (0,1), (2,3), (4,5)

```
if (arr[j] > arr[j + 1])
    swap(arr[j], arr[j + 1]);
```

🔸 **Odd Phase**

```
#pragma omp parallel for
for (int j = 1; j < n - 1; j += 2)
```

👉 Compare pairs (1,2), (3,4), (5,6)

👉 This method is called **Odd-Even Transposition Sort**

🔴 **Merge Function**

```
void merge(vector<int>& arr, int l, int m, int r)
```

👉 Combines two sorted parts

```
vector<int> L(n1), R(n2);
```

👉 Create left and right temporary arrays

```
while (i < n1 && j < n2)
```

👉 Compare elements from both arrays

```
if (L[i] <= R[j])
    arr[k++] = L[i++];
else
    arr[k++] = R[j++];
```

👉 Insert smaller element into main array

```
while (i < n1) ...
while (j < n2) ...
```

👉 Copy remaining elements

🔴 **Sequential Merge Sort**

```
void mergeSortSeq(vector<int>& arr, int l, int r)
```

```
if (l < r)
```

👉 Base condition

```
int m = (l + r) / 2;
```

👉 Find middle

```
mergeSortSeq(arr, l, m);
mergeSortSeq(arr, m + 1, r);
```

👉 Recursively divide array

```
merge(arr, l, m, r);
```

👉 Merge sorted halves

🔴 **Parallel Merge Sort**

```
void mergeSortParallel(vector<int>& arr, int l, int r)
```

```
if (l < r)
```

👉 Continue splitting

```
#pragma omp parallel sections
```

👉 Execute sections in parallel

```
#pragma omp section
mergeSortParallel(arr, l, m);
```

👉 Left half sorted in one thread

```
#pragma omp section
mergeSortParallel(arr, m + 1, r);
```

👉 Right half sorted in another thread

```
merge(arr, l, m, r);
```

👉 Merge results

🔹 **Main Function**

```
vector<int> arr(SIZE), temp;
```

👉 Create main array and temporary array

```
srand(time(0));
generateRandom(arr);
```

👉 Initialize and fill random values

🔹 Timing Sequential Bubble

```
temp = arr;
start = omp_get_wtime();
bubbleSortSeq(temp);
end = omp_get_wtime();
```

👉 Measure time

🔹 Timing Parallel Bubble

Same process

🔹 Timing Sequential Merge

Same process

🔹 Timing Parallel Merge

```
mergeSortParallel(temp, 0, SIZE - 1);
```

🔹 Print Results

```
cout << "Time: " << (end - start)
```

👉 Display execution time

🎯 Final 20-Second Explanation (Use This 💯)

“This program compares sequential and parallel versions of Bubble Sort and Merge Sort. OpenMP is used to parallelize loops in Bubble Sort and divide work in Merge Sort using sections. Execution time is measured using omp_get_wtime to analyze performance improvement.”

⚠️ Important Viva Line

“Merge Sort shows better parallel performance because it divides the problem into independent subproblems, while Bubble Sort has dependencies.”

We'll use a **small array** (instead of 10000):

arr = [5, 3, 1, 4, 2]

🔵 1. Sequential Bubble Sort (Code + Dry Run)

Code:

```
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        if (arr[j] > arr[j + 1])
            swap(arr[j], arr[j + 1]);
    }
}
```

🔹 Pass 1 (i = 0)

Comparisons:

(5,3) → swap → [3,5,1,4,2]
(5,1) → swap → [3,1,5,4,2]
(5,4) → swap → [3,1,4,5,2]
(5,2) → swap → [3,1,4,2,5]

🔹 Pass 2 (i = 1)

(3,1) → swap → [1,3,4,2,5]
(3,4) → no swap
(4,2) → swap → [1,3,2,4,5]

🔹 Pass 3 (i = 2)

(1,3) → no swap
(3,2) → swap → [1,2,3,4,5]

🔹 Final Output:

[1,2,3,4,5]

🔵 2. Parallel Bubble Sort (Code + Dry Run)

Code:

```
#pragma omp parallel for
for (int j = 0; j < n - 1; j += 2)

#pragma omp parallel for
for (int j = 1; j < n - 1; j += 2)
```

🔹 Iteration 1

Even Phase:

Compare (0,1) and (2,3)

(5,3) → swap → [3,5,1,4,2]
(1,4) → no swap

Odd Phase:

Compare (1,2) and (3,4)

(5,1) → swap → [3,1,5,4,2]
(4,2) → swap → [3,1,5,2,4]

🔹 Iteration 2

Even phase:

(3,1) → swap → [1,3,5,2,4]
(5,2) → swap → [1,3,2,5,4]

Odd phase:

(3,2) → swap → [1,2,3,5,4]
(5,4) → swap → [1,2,3,4,5]

✅ Final:

[1,2,3,4,5]

🧠 Key Point:

"Even and odd pairs are processed in parallel to avoid conflicts."

🔴 3. Sequential Merge Sort (Code + Dry Run)

Code:

```
int m = (l + r) / 2;
mergeSortSeq(left);
mergeSortSeq(right);
merge(...);
```

🔹 Step 1: Divide

[5,3,1,4,2]
→ [5,3] [1,4,2]
→ [5] [3] [1] [4] [2]

🔹 Step 2: Merge

[5] + [3] → [3,5]
[4] + [2] → [2,4]
[1] + [2,4] → [1,2,4]

🔹 Final Merge

[3,5] + [1,2,4]
→ [1,2,3,4,5]

🔴 4. Parallel Merge Sort (Code + Dry Run)

Code:

```
#pragma omp parallel sections

#pragma omp section
left half

#pragma omp section
right half
```

🔹 What Happens

Instead of:

Left → then Right

👉 It becomes:

Left || Right (parallel execution)

🔹 **Example:**

Thread 1 → sort [5,3]
Thread 2 → sort [1,4,2]

Thread 1:

[5,3] → [3,5]

Thread 2:

[1,4,2] → [1,2,4]

Final Merge:

[3,5] + [1,2,4]
→ [1,2,3,4,5]

⏱️ Timing Part (Important)

Code:

```
start = omp_get_wtime();
...
end = omp_get_wtime();
```

👉 Measures execution time of each algorithm

🎯 Final Viva Explanation (Use This 💯)

"Sequential Bubble Sort compares elements one by one, while parallel version uses odd-even phases to process pairs simultaneously. Merge Sort divides the array recursively, and in parallel version both halves are processed simultaneously using OpenMP sections. This reduces execution time."

⚠️ Important Line (Don't Miss)

"Parallel Merge Sort is more efficient because it divides work into independent tasks, while Bubble Sort has dependencies."

🧠 Final Summary

Algorithm	Parallel Benefit
Bubble Sort	Low
Merge Sort	High

▶️ How to Compile & Run

```
g++ yourfile.cpp -fopenmp -O2 -o sort
./sort
```

👉 -fopenmp → enables parallelism
👉 -O2 → improves performance (important for timing)

🧪 What Happens Internally

1. **Array of size 10000** is created
2. It is filled with **random numbers**
3. Same data is copied into temp before each sort
4. Four algorithms run:
    - Sequential Bubble Sort
    - Parallel Bubble Sort
    - Sequential Merge Sort
    - Parallel Merge Sort
5. Time is measured using:

```
omp_get_wtime()
```

✅ Expected Output (Example)

```
Sequential Bubble Sort Time: 2.35 sec
Parallel Bubble Sort Time: 1.40 sec
Sequential Merge Sort Time: 0.010 sec
Parallel Merge Sort Time: 0.005 sec
```

🔍 How to Interpret Output

🔵 Bubble Sort

- Parallel is **faster than sequential**
- But still **slow overall**

👉 Why?

Bubble sort has dependency between elements

🔴 **Merge Sort**

- Very fast compared to Bubble Sort
- Parallel version is **even faster**

👉 Why?

Divide-and-conquer allows true parallel execution

⚠️ **Important Observations (Say in Viva)**

✓ **1. Bubble Sort Improvement is Limited**

"Due to dependency between elements, parallelism is limited."

✓ **2. Merge Sort Shows Better Speedup**

"Merge Sort divides problem into independent subproblems."

✓ **3. Timing May Vary**

👉 Every run gives slightly different time
👉 Because:

- CPU load
- Thread scheduling

🚨 **If Output Looks Wrong**

❌ **If parallel is slower**

👉 Happens for small inputs
👉 Because:

"Thread overhead > computation time"

❌ **If times are almost same**

👉 Try increasing:

#define SIZE 50000

🔥 What To Say After Running (IMPORTANT)

“The results show that parallel algorithms reduce execution time. However, the improvement depends on the nature of the algorithm. Merge Sort benefits more from parallelism compared to Bubble Sort.”

🧠 Quick Self-Check

✓ Program runs without error
✓ All 4 timings printed
✓ Parallel < Sequential (at least for Merge Sort)

💡 Pro Tip (Marks Booster)

Run twice and say:

“Results vary slightly due to system scheduling, but trend remains same.”

🎯 Basic Concepts

❓ 1. What is sorting?

Answer:
Sorting is the process of arranging elements in a specific order, usually ascending or descending.

❓ 2. What is Bubble Sort?

Answer:
Bubble Sort is a simple comparison-based algorithm that repeatedly swaps adjacent elements if they are in the wrong order.

❓ 3. Time complexity of Bubble Sort?

Answer:
Worst and average case: **$O(n^2)$**
Best case: **$O(n)$** (if already sorted)

❓ 4. What is Merge Sort?

Answer:
Merge Sort is a divide-and-conquer algorithm that divides the array into halves, sorts them recursively, and merges them.

❓ 5. Time complexity of Merge Sort?

Answer:
All cases: **$O(n \log n)$**

⚙️ Parallel Computing Questions

❓ 6. What is OpenMP?

Answer:
OpenMP is an API used for parallel programming in C/C++ that enables multi-threaded execution on shared memory systems.

❓ 7. What does #pragma omp parallel for do?

Answer:
It divides loop iterations among multiple threads so they execute simultaneously.

❓ 8. What is #pragma omp parallel sections?

Answer:
It allows different blocks of code (sections) to run in parallel using separate threads.

❓ 9. What is a thread?

Answer:
A thread is a lightweight unit of execution within a process.

❓ 10. What is parallelism?

Answer:
Parallelism means executing multiple tasks at the same time using multiple processors or cores.

🔵 Bubble Sort Specific

❓ 11. How is Bubble Sort parallelized?

Answer:
Using **Odd-Even Transposition Sort**, where even and odd indexed pairs are compared in parallel.

❓ 12. Why normal Bubble Sort cannot be directly parallelized?

Answer:
Because each comparison depends on previous swaps, creating **data dependency**.

❓ 13. What is Odd-Even Transposition Sort?

Answer:
It alternates between comparing even indexed pairs and odd indexed pairs, allowing safe parallel execution.

🔴 Merge Sort Specific

❓ 14. Why is Merge Sort suitable for parallelization?

Answer:
Because it divides the problem into **independent subproblems** that can be solved simultaneously.

❓ 15. How is Merge Sort parallelized?

Answer:
Using #pragma omp parallel sections to process left and right halves concurrently.

❓ 16. What is the role of merge function?

Answer:
It combines two sorted subarrays into a single sorted array.

⏱️ Performance Questions

❓ 17. How do you measure execution time?

Answer:
Using:

omp_get_wtime()

❓ 18. Which algorithm performs better in parallel?

Answer:
Merge Sort performs better than Bubble Sort.

❓ 19. Why Parallel Bubble Sort is still slow?

Answer:
Because of **O(n²)** complexity and dependencies between elements.

❓ 20. Why Parallel Merge Sort is faster?

Answer:
Because it uses divide-and-conquer and allows independent parallel execution.

⚠️ Important Conceptual Questions

❓ 21. What is synchronization overhead?

Answer:
Extra time required to manage multiple threads accessing shared resources.

❓ 22. What is scalability?

Answer:
Ability of a program to improve performance as the number of processors increases.

❓ 23. What happens if input size is small?

Answer:
Parallel version may be slower due to **thread creation overhead**.

❓ 24. Why do we use large input size (10000)?

Answer:
To clearly observe performance difference between sequential and parallel algorithms.

❓ 25. What is speedup?

Answer:
Speedup = Sequential Time / Parallel Time

🚀 Advanced (High Chance Questions)

❓ 26. Which is better: Bubble Sort or Merge Sort?

Answer:
Merge Sort, because it has lower complexity (**O(n log n)**).

❓ 27. What are advantages of parallel algorithms?

Answer:

- Faster execution
- Efficient use of CPU cores
- Suitable for large data

❓ 28. Limitations of your program?

Answer:

- Bubble Sort has limited parallelism
- Thread overhead
- Performance depends on hardware

🎯 Final Killer Answer (Say This 💯)

If examiner asks:
👉 *"What is your conclusion?"*

Answer:

"Parallel algorithms reduce execution time using multiple threads. However, the performance improvement depends on the algorithm. Merge Sort benefits more due to independent subproblems, while Bubble Sort has limited parallelism due to dependencies."

🔥 Pro Tip

If you confidently say:

"Merge Sort is more scalable and efficient for parallel execution."

👉 You instantly stand out